

Mukta Mishta

EE 446

Assignment 1

Part 1

Can you **further reduce the model size** beyond the smallest model obtained in parts **(b)**, **(c)**, or **(d)**, **without sacrificing significant classification performance**?

Yes, the model size can be reduced further by combining multiple compression techniques together, such as applying INT8 quantization to the smaller knowledge-distilled student model. This reduces the number of parameters and stores them using fewer bits, creating a smaller TFLite model. However, additional compression may slightly reduce accuracy, so there is a tradeoff between model size and performance.

Your task is to:

1. **Analyze and compare** the results from previous parts: Which model had the smallest size? Which performed best?

Among all models, the INT8 quantized model had the smallest size at 5.74 KB and also achieved the best performance with 100% accuracy on the test set. The baseline, dynamic range quantized, float16 quantized, and pruned models all achieved about 98.15% accuracy, while the knowledge distillation student model achieved slightly lower accuracy at 96.30%. Overall, INT8 quantization provided the best balance between small model size and high classification performance.

2. **Propose a strategy** that combines or enhances techniques learned so far.

A good strategy is to combine knowledge distillation with INT8 quantization. First, train a smaller student model using the baseline model as the teacher, then apply full integer quantization to the student model. This should reduce size more than using the baseline model alone, while still keeping reasonable accuracy.

4. **Implement** your proposed solution.
5. **Evaluate** the resulting model using both:
 - TFLite model size (in KB)
 - Classification performance (accuracy and report)

6. Justify your results:

- If further size reduction is not possible without major loss of accuracy, explain why.
- If you succeed in reducing the size further, highlight what change made the biggest difference.

I was able to reduce the model size further by combining knowledge distillation with INT8 quantization. The biggest improvement came from using a smaller student model before applying quantization, which reduced the number of parameters while still keeping good accuracy. The final KD + INT8 model reached a size of 3.62 KB with 96.30% accuracy, showing that additional compression is possible with only a small decrease in performance.

Part 2

1. The free-tier version of Edge Impulse supports several types of learning blocks, including supervised learning, unsupervised learning, and transfer learning workflows. Supervised learning can be used for classification and regression tasks, such as image classification using MobileNet or audio keyword spotting using a neural network classifier. Unsupervised learning can include anomaly detection and clustering-based approaches. Transfer learning is also supported for image and audio applications using pretrained neural networks.
2. Under the “Classifier” section, the free-tier Edge Impulse account includes several layer types such as Dense layers, Convolutional layers, Depthwise Convolutional layers, Pooling layers, Dropout layers, Flatten layers, and Softmax output layers. Dense layers perform general feature learning, convolutional layers extract spatial or temporal patterns, pooling layers reduce feature-map size, dropout layers help prevent overfitting, flatten layers convert multidimensional data into vectors, and softmax layers generate classification probabilities.³
3. The Deployment section in the free-tier Edge Impulse supports multiple model compression methods, including quantization (such as INT8 quantization), pruning/sparsity optimization, EON Compiler optimization, and TensorFlow Lite conversion. These techniques reduce model size, memory usage, and inference latency for embedded deployment.
4. The free-tier version of Edge Impulse supports synthetic data generation for image, audio, and sensor/time-series data. For image data, augmentation methods include

rotation, flipping, scaling, cropping, and noise injection. Audio data supports augmentation such as background noise addition, time shifting, and pitch adjustments. Time-series sensor data can include noise injection and signal transformations. Synthetic data is important because it increases dataset diversity, improves model robustness, reduces overfitting, and helps compensate for limited real-world training data.